

Doc. no.:	P3060R3
Date:	2025-6-18
Audience:	LWG
Reply-to:	Weile Wei <weilewei09@gmail.com> Zhihao Yuan <zy@miator.net>

Add std::views::indices(n)

- Add std::views::indices(n)
 - Revision History
 - Abstract
 - Motivation
 - Implementation and Usage
 - Prior Art
 - Technical Decisions
 - Wording
 - Feature test macro
 - References

Revision History

Since R0

- Move `upto` from the `std::ranges` namespace into `std::views`
- Use a more convincing example
- Incorporate with the current `iota` wording in the standard

Since R1

- Change the name `views::upto` to `views::indices` .
- Use the `is-integer-like` exposition-only constraint rather than the `std::integral` (`is-integer-like` allows implementation-defined types and removes `bool`).

Since R2

- Feature test macro

Abstract

We propose adding `std::views::indices(n)` to the C++ Standard Library as a range adaptor that generates a sequence of integers from `0` to `n-1`.

Motivation

Currently, `iota(0, ranges::size(rng))` does not compile due to mismatched types (see point 1 in Background section). Then, users need to write a workaround like

`iota(range_size_t<decltype(rng)>{}, ranges::size(rng))`, which is not straightforward and cumbersome. An example illustrating this issue is available at [x8nWxqE9v](https://godbolt.org/z/x8nWxqE9v)

Compiler Explorer
(<https://godbolt.org/z/x8nWxqE9v>):

C++23	<pre>std::vector rng(5, 0); // auto res1 = views::iota(0, ranges::size(rng)); // does not compile auto res2 = iota(range_size_t<decltype(rng)>{}, ranges::size(rng)); std::print("{} ", res2); // [0, 1, 2, 3, 4]</pre>
P3060	<pre>std::vector rng(5, 0); std::print("{} ", views::indices(ranges::size(rng))); // [0, 1, 2, 3, 4]</pre>

`std::views::indices(n)` eases this pattern by providing a straightforward method to generate integer sequences, improving readability and killing arcane code.

Implementation and Usage

```
namespace std::views {
    inline constexpr auto indices = [] < /*is-integer-like*/ I> (I n) {
        return std::views::iota(I{}, n);
    };
}
```

Usage:

```
int main() {
    std::vector rng(5, 0);
    std::print("{} ", views::indices(ranges::size(rng))); // [0, 1, 2, 3, 4]
}
```

Compiler Explorer

Implementation: [hsP97jKzv](https://godbolt.org/z/hsP97jKzv) (https://godbolt.org/z/hsP97jKzv)

Prior Art

Two preceding proposals have provided foundation for `std::views::indices(n)` :

1. *P2214R2: A Plan for C++26 Ranges*^[1] highlights the issue with `views::iota(0, r.size())` not compiling due to mismatched types. `std::ranges::views::iota` requires both arguments to be of the same type, or at least commonly comparable. This becomes problematic when comparing `int` (often 32-bit) with `std::size_t` (often 64-bit), which is usually wider on 64-bit systems. The same example from above Motivation section can be viewed at [x8nWxqE9v](https://godbolt.org/z/x8nWxqE9v) (https://godbolt.org/z/x8nWxqE9v).
Compiler Explorer

```
std::vector rng(5, 0);
auto res1 = iota(0, ranges::size(rng)); // does not compile
```

2. *P1894R0: Proposal of std::upto, std::indices and std::enumerate*^[2] proposed an implementation (see below) that our proposal refines. Our approach offers a more consistent interface, fitting seamlessly with existing standard library.

```
// std::upto implementation example
// P1894R0: Proposal of std::upto, std::indices and std::enumerate
namespace std {
    template<typename Int>
    constexpr auto upto(Int&& i) noexcept {
        return std::ranges::iota_view{Int(), std::forward<Int>(i)};
    }
}
```

Technical Decisions

1. Limiting to `is-integer-like` : `is-integer-like` allows implementation-defined types and removes `bool`.
2. Lambda-based Approach: Using a lambda is consistent with the established range adaptor patterns. Moreover, the use of `constexpr` allows for the evaluation to occur at compile-time.
3. Leveraging Existing `iota_view` Instead of Creating a New `indices_view` : Introducing `indices_view` would mean adding a component similar to what already exists, causing confusion and maintainability issues for users. By leveraging `iota_view` , we simplify the implementation and reuse of what the current C++ Standard Library offers. Additionally, any future optimizations to `iota_view` will automatically benefit `std::views::indices` . Therefore, by extending `iota_view` , `std::views::indices` becomes more maintainable, efficient, and simple.

Wording

Add a new paragraph under `[range.iota.overview]` (<https://eel.is/c++draft/range.iota.overview>):

26.6.4.1 Overview `[range.iota.overview]` (<https://eel.is/c++draft/range.iota.overview>)

The name `views::indices` denotes a customization point object (`[customization.point.object]` (<https://eel.is/c++draft/customization.point.object>)). Given subexpression `E` , let `T` be `remove_cvref_t<decltype((E))>` . `views::indices(E)` is expression-equivalent to `views::iota(T(0), E)` if `T` models *is-integer-like* , and ill-formed otherwise.

Modify `[ranges.syn]` (<https://eel.is/c++draft/ranges.syn>) as follows:

```
// [range.iota], iota view
[...]
namespace views {inline constexpr unspecified iota = unspecified; }
    inline constexpr unspecified iota = unspecified;
    inline constexpr unspecified indices = unspecified;
}
```

Feature test macro

Add `__cpp_lib_ranges_indices` to [version.syn] (<https://eel.is/c++draft/version.syn>):

```
#define __cpp_lib_ranges_indices 202XXXL // also in <ranges>
```

References

1. P2214R2 *A Plan for C++26 Ranges*. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r0.html>) ↩
2. P1894R0 *Proposal of `std::upto`, `std::indices` and `std::enumerate`*. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1894r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1894r0.pdf>) ↩